TP8 — Food Store en la feria: la base de la tablet y el cierre de la capa de datos

Base de Datos II · UTN TUP · Equipo: Agustina Fontagnol, Gianella Peña, Martina Suarez · 4 de noviembre de 2026

Todo con datos de prueba y sobre la copia `foodstore_desarrollo` (servidor) y `movil/foodstore_movil.db` (tablet). Los scripts están en `movil/` y `db/`; este informe solo muestra lo que se pide.

1.a Qué datos viajan

Columna de `productos`	Viaja	Cómo llega a la tablet
`id`, `nombre`	Sí	tal cual
`precio`	Sí	`precio_centavos` (entero)
`stock`	Sí	`stock_visto` (lo que vio la tablet, no el real)
`disponible`, `eliminado`	Sí	0 o 1; las bajas lógicas viajan como lápidas
`descripcion`, `imagen`, `categoria_id`	No	no hacen falta para vender en la feria
`created_at`, `updated_at`, `deleted_at`	No	auditoría del servidor
Tabla `clientes` (contraseña, mail, celular)	No	datos personales: nunca salen del servidor

Viaja lo mínimo para mostrar el catálogo y armar un borrador sin señal. PostgreSQL es la fuente de verdad de precio, stock y ventas: la tablet solo guarda una copia de lectura y propone pedidos.

2.c Los casos

Fecha de la copia (`servidor_snapshot.csv`): 2026-11-03 11:19:48-03 (10 productos exportados de `v_snapshot_movil`). En cada caso, `python movil/simulador_casos.py` cambia datos **en PostgreSQL** (`foodstore_desarrollo`): abre una transacción, aplica el `UPDATE`, exporta `v_snapshot_movil` a un CSV nuevo desde esa misma transacción y hace `ROLLBACK`; después importa el CSV en `snapshot_servidor` y corre `semaforo.sql`. Los identificadores son UUID v4 (el esquema lo exige con un `CHECK`); abajo se abrevian con los primeros 8 caracteres.

C2 — sube el precio. Borrador de 2 unidades del producto 1 a 12063 centavos; el servidor sube el producto 1 a 14000.

```
-- PostgreSQL: BEGIN; UPDATE productos SET precio = 140.00 WHERE id = 1;  (exportar v_snapshot_movil;
ROLLBACK)
-- a8b5fbe6… | CONFLICTO | Precio cambió en producto 1: visto=12063, actual=14000
UPDATE item_borrador SET precio_centavos_visto = 14000 WHERE id_local = 'c1abcbf0-…' AND producto_id = 1;
INSERT INTO cola_sync (solicitud_uuid, id_local, estado) VALUES ('4e790513-…', 'c1abcbf0-…', 'PENDIENTE');
-- 4e790513… | SIMULADO_APROBADO
```

El semáforo encontró el precio distinto y no había rojo, así que dio amarillo. El cliente aceptó, se actualizó el borrador y la solicitud nueva dio verde; la primera queda como CONFLICTO. La segunda solicitud se admite porque la anterior ya no está PENDIENTE (índice único parcial `ux_cola_sync_una_pendiente`).

C3 — más unidades de las que quedan. La tablet ve 61 unidades del producto 2 y el borrador pide 50 (válido con la copia local); el servidor baja el stock a 10.

```
-- PostgreSQL: BEGIN; UPDATE productos SET stock = 10 WHERE id = 2;  (exportar v_snapshot_movil; ROLLBACK)
-- a179e489… | RECHAZADO | Stock insuficiente en producto 2: pedido=50, stock=10
```

`cantidad > stock_visto` de la instantánea nueva es un rojo: el pedido no se puede cumplir aunque la tablet no se haya enterado.

C4 — producto dado de baja. Borrador con 2 unidades del producto 3; el servidor da de baja el producto 3.

```
-- PostgreSQL: BEGIN; UPDATE productos SET eliminado = TRUE WHERE id = 3;  (exportar v_snapshot_movil;
ROLLBACK)
-- 86f12151… | RECHAZADO | Producto 3 dado de baja
-- catálogo disponible: ids 1, 2, 4 a 10   |   producto_cache id=3: eliminado=1 (la fila queda)
```

La baja es un rojo. Al recargar, `WHERE disponible = 1 AND eliminado = 0` ya no lista el producto, pero la fila sigue en `producto_cache` porque el borrador la referencia.

C5 — sin cambios. Borrador de 2 unidades del producto 4 a 17272, igual al servidor.

```
-- 38f3809c… | SIMULADO_APROBADO
```

Todo coincide: verde. No es una venta; solo el servidor puede confirmarla y descontar stock.

C6 — reenviar la misma solicitud.

```
INSERT INTO cola_sync (solicitud_uuid, id_local, estado) VALUES ('38f3809c-…', 'cb420e06-…', 'PENDIENTE');
-- Error: UNIQUE constraint failed: cola_sync.solicitud_uuid
-- PostgreSQL: BEGIN; UPDATE productos SET precio = 9.99 WHERE id = 4;  (exportar; ROLLBACK)  y repetir el
semáforo
-- 38f3809c… | SIMULADO_APROBADO (sin cambios)
```

El UUID repetido choca con la clave primaria y no se duplica; como el semáforo solo toma las PENDIENTE, repetirlo no reevalúa una solicitud ya resuelta. En un sistema real la protección iría en el servidor (parada 8): una tabla de solicitudes procesadas con el UUID como clave única, que devuelva el resultado anterior en lugar de volver a confirmar la venta.

Registro de evaluaciones. Cada resolución del semáforo queda en `evaluacion_sync` (estado, motivo y fecha de la instantánea usada), cargada por un trigger. Tras los casos hay 6 filas para 6 solicitudes: repetir el semáforo en C6 no agregó ninguna.

PostgreSQL no cambió. Consulta de control de `db/vista_snapshot.sql` sobre `foodstore_desarrollo`, que el simulador corre antes y después de los casos:

Valor en PostgreSQL	Antes	Después	¿Cambió?
Pedidos (`pedidos`)	200000	200000	No
Detalles (`detalle_pedido`)	600000	600000	No
Stock total de los productos 1 a 10	553	553	No
Suma de precios de los productos 1 a 10	2646.99	2646.99	No
Productos 1 a 10 dados de baja	0	0	No
Huella md5 (id, precio, stock, eliminado)	`23b70cf6f125e3adec95513c054359b3`	`23b70cf6f125e3adec95513c054359b3`	No

Los cambios de precio, stock y baja de C2, C3, C4 y C6 se hicieron en PostgreSQL dentro de `BEGIN` y terminaron en `ROLLBACK`, por eso la huella es idéntica. La tablet nunca escribe en el servidor.

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda
PostgreSQL (`foodstore_desarrollo`)	catálogo completo, precios, stock, clientes, pedidos y su detalle	**Es la fuente de verdad** de precio, stock y ventas; confirma o rechaza
MongoDB (`foodstore_u7.pedidos`, Unidad 7)	copia documental de los pedidos 1 a 10 con líneas y cliente embebidos	PostgreSQL; MongoDB es una copia para consultar, no decide nada
SQLite (`foodstore_movil.db`, tablet)	`producto_cache` (copia de lectura), borradores, ítems, `cola_sync` y su registro `evaluacion_sync`	PostgreSQL manda sobre el catálogo; sobre los borradores manda la tablet, pero no valen como venta

```
PostgreSQL ──migración──► MongoDB        (pedidos, solo lectura)
PostgreSQL ──snapshot CSV─► SQLite       (catálogo)
SQLite ──solicitud (UUID)─► PostgreSQL   (propone; el servidor valida y decide)
```

3.b Plan de mantenimiento (agregado al plan de la Unidad 6)

¿Qué se respalda y cómo se prueba la restauración?

- **PostgreSQL** (crítico): `pg_dump -Fc` con el script de la Unidad 6. La restauración se prueba en `foodstore_restore_test` comparando conteos de tablas, como en el simulacro de U6.
- **MongoDB** (`foodstore_u7`): `mongodump` de `pedidos`; se prueba con `mongorestore` en otra base y `countDocuments()` igual a 10. Si se pierde, se regenera desde PostgreSQL con `migrar_pedidos.js`.
- **SQLite**: no se respalda el archivo; se recrea con `schema_sqlite.sql` + `carga_snapshot.sql` y se prueba con `pruebas_sqlite.sql`. Lo único que no se recupera son los borradores sin sincronizar, por eso se envían al terminar cada día de feria.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? Pueden tener una copia más nueva que el servidor, con stock y precios que ya no existen allá. No confían en su cache: bajan una instantánea nueva completa (reemplaza `producto_cache`), crean una solicitud nueva (UUID nuevo) por cada borrador sin confirmar y vuelven a correr el semáforo, así lo que ya no coincide queda en CONFLICTO o RECHAZADO. Las ventas confirmadas después del respaldo se concilian aparte, desde el servidor.

¿Cada cuánto y quién revisa CONFLICTO y RECHAZADO? Cada vendedor, en su tablet, antes de reconectar y al cierre de cada día de feria (decide: aceptar el precio nuevo, achicar la cantidad o descartar); el historial queda en `evaluacion_sync`. El supervisor revisa semanalmente los pedidos que quedaron sin resolver.

3.c Code review con IA

Alcance. OpenCode (agente Plan) revisó `parcial1_02` a `parcial1_10` (hallazgos 1 a 5). La revisión se completó con Claude Code sobre el resto de los scripts de la cursada: `parcial1_01`, los 6 anexos de TP5 (`db/anexos_tps/`), los 8 scripts de `sql/` (U5 y U6: roles, anonimización, auditoría, simulacro, respaldo, restauración, health check y mantenimiento), las copias de `entrega_tp6/sql` y los 5 de `entrega_tp7` (hallazgos 6 a 14). El inventario completo por unidad (U1 a U8, 35 archivos) está en `docs/inventario_scripts.md`. Las pruebas de PostgreSQL se corrieron en una copia descartable de `foodstore_desarrollo` (`foodstore_tp8_verif`, restaurada de un `pg_dump` y ya eliminada), porque `parcial1_03`, `_07` y `_08` no están aplicados en la base de trabajo; las de MongoDB, en una colección temporal.

#	Unidad	Hallazgo	Decisión	Motivo y cómo se comprobó en la base
1	U3 Transacciones	Posible race condition en `sp_crear_pedido` por `FOR UPDATE` dentro del loop	**Descartado**	`GROUP BY ... ORDER BY 1` ordena siempre igual los bloqueos, así que no hay deadlock. Con stock 1 en el producto 7 y dos sesiones: la B queda esperando (`wait_event_type = Lock`, `wait_event = transactionid`); al hacer COMMIT la A, la B falla con `Stock insuficiente para 7: pedido 1, disponible 0`. Stock final 0 y un solo pedido nuevo
2	U2 Índices	El índice parcial `idx_productos_categoria_eliminado` no se usa sin `eliminado = FALSE`	**Corregido** (ya mitigado)	Sin el predicado el plan es `Seq Scan` aun con `enable_seqscan = off`; con `eliminado = FALSE` usa un índice parcial: `idx_tp4_prod_cat` (de TP4) y, al quitarlo, `idx_productos_categoria_eliminado`. La vista `v_productos_vigentes_con_categoria` ya incluye el filtro
3	U4 Vistas	Agregar `CHECK (total_facturado >= 0)` a `mv_facturacion_categoria_mes`	**Descartado**	`CREATE MATERIALIZED VIEW ... CHECK` da error de sintaxis en o cerca de «CHECK»; PostgreSQL solo admite CHECK en tablas. `chk_detalle_subtotal` rechaza un INSERT con subtotal `-200.00` y la vista tiene 0 filas negativas de 65

4	U6 Seguridad	Contraseñas en texto plano (`'hash_de_prueba'`) en los datos de prueba de `parcial1_09` y `parcial1_10`	**Aceptado y corregido**	El riesgo es real: el patrón puede copiarse. En ambos scripts se reemplazó por un hash falso con formato bcrypt y un comentario de seguridad. `grep hash_de_prueba db/*.sql` no da resultados y los dos scripts corren completos en la copia (los errores que muestran son las demostraciones del propio script)
5	U2 Índices	Falta un índice sobre `pedidos.fecha` para consultas por rango	**Descartado**	Rango de enero de 2026 (3925 pedidos): aun creando un índice solo sobre `fecha`, el plan sigue usando `idx_tp4_q1_fast (estado, fecha)` y no el nuevo (4,5 ms). Sin los índices de TP4, el rango lo resuelve `idx_pedidos_estado_eliminado_fecha` (Bitmap Index Scan, 3925 filas, ninguna descartada)
6	Parcial 1 (DDL)	`clientes.rol` es `VARCHAR(50)` sin restricción	**Aceptado** (a aplicar)	Un INSERT con `rol = 'CUALQUIER_COSA'` y otro con `'HACKER'` se aceptan (ROLLBACK después). Corrección: `CHECK (rol IN (...))` con los roles reales o un tipo enumerado
7	Parcial 1 (DDL)	`chk_cliente_mail` (`POSITION('@' IN mail) > 1`) es débil	**Descartado**	Acepta `x@` y `a@@b` (comprobado con INSERT y ROLLBACK). Es una red mínima a propósito: validar el formato completo con una regex en la base rechazaría mails válidos y esa validación corresponde a la aplicación
8	Parcial 1 (DML)	`parcial1_04` dice «Variante B (desactivada)» pero `TRUNCATE categorias ... CASCADE` (línea 43) está sin comentar	**Aceptado** (a aplicar)	Contradice el encabezado («se conservan categorías, no se truncan»). En la copia, dentro de BEGIN/ROLLBACK: `categorias` pasa de 5 a 0 y el `CASCADE` también vacía `productos` y `detalle_pedido` (50000 productos antes). Corrección: comentar el bloque de la variante B
9	Parcial 1 (consultas)	El top 10 de productos más vendidos (`ORDER BY unidades_vendidas DESC LIMIT 10`) no desempata	**Aceptado** (a aplicar)	En la base los puestos 9 a 12 tienen 60 unidades cada uno (ids 31380, 40800, 29020, 40140), así que qué producto entra en el top 10 queda al azar del plan. Corrección: agregar `, pr.id` al `ORDER BY`
10	U5 Roles	`ALTER DEFAULT PRIVILEGES FOR ROLE admin_datos ... GRANT SELECT ON TABLES TO rol_reportes` contradice «solo vistas, nunca tablas base»	**Aceptado** (a aplicar)	Reproducido en una transacción: `admin_datos` crea una tabla nueva y `has_table_privilege('rol_reportes', tabla, 'SELECT')` da `t`, así que cualquier tabla futura (por ejemplo con datos de clientes) queda legible. Corrección: quitar ese `ALTER DEFAULT PRIVILEGES` y otorgar permisos por objeto
11	U5 Auditoría	`auditoria_test.sql` pide activar `log_failed_auth = on` (PG 14+)	**Aceptado** (a aplicar)	`SHOW log_failed_auth` da *parámetro de configuración «log_failed_auth» no reconocido*: no existe en PostgreSQL 17. Los fallos de autenticación se registran siempre como FATAL; corrección: quitar la línea del encabezado

12	U7 MongoDB	`verificacion_integridad.js`, R1b: imprime `...toArray().length` como cantidad de líneas inconsistentes	**Aceptado** (a aplicar)	Con `$count`, el arreglo tiene 0 o 1 elementos. En una colección temporal con 3 líneas inconsistentes el script imprimiría 1 (el valor correcto, `r[0].n`, es 3). Hoy da 0 y es correcto, pero no detectaría la magnitud de un error. Corrección: leer `r[0]?.n ?? 0`
13	TP5 anexos	`parte5_competencia.sql` usa `producto`, `id_categoria` e `id_producto`, que no existen en `foodstore`	**Descartado**	No es un error del script: la tabla `producto` existe en `foodstore_tpi`, la base del TP3 a la que está dirigido; en `foodstore_desarrollo` solo existe `productos`. Cada bloque ya está envuelto en BEGIN/ROLLBACK
14	U6 Respaldo	`sql/` y `entrega_tp6/sql/` tienen los mismos 4 scripts duplicados	**Descartado**	`cmp` confirma que los 4 archivos son idénticos; la copia en `entrega_tp6` es parte de la entrega de esa unidad y mantenerlas iguales es más barato que referenciar rutas externas

Revisados sin otros hallazgos: `parcial1_02` a `_10` (más allá de los hallazgos 1 a 5), `usuario_anon.sql`, `simulacro_incidente.sql`, `health_check.sql`, `maintenance.sql`, `backup_foodstore.ps1`, `restore_test.ps1`, `benchmark_vista_materializada.sql`, `tp5_mediciones_pendientes.sql`, `verificacion_vistas_parteB.sql`, `vistas_reportes_*` y los scripts de `entrega_tp7` salvo el R1b.